

Lecture 7: Arithmetic Logic Unit (ALU)

Learning Objectives

By the end of this lecture, students will be able to:

- Define the Arithmetic Logic Unit (ALU).
 - Explain the purpose of the ALU.
 - Differentiate between arithmetic and logical operations.
 - Understand comparison and shift operations.
 - Explain the concept of status flags.
 - Understand how the ALU works with registers and the Control Unit.
 - Relate ALU operations to Assembly Language.
-

1. Introduction

Teaching Script

First 30-Second SEO Phrase

Naturally mention:

"In today's Computer Organization and Assembly Language lecture, we'll learn the Arithmetic Logic Unit (ALU), including arithmetic operations, logical operations, comparison operations, shift operations, and CPU status flags with simple examples."

In the previous lecture, we studied the internal organization of the CPU and learned that the CPU mainly consists of the Control Unit, Arithmetic Logic Unit (ALU), Registers, and Cache Memory.

Today, we'll focus on one of the most important components inside the CPU—the Arithmetic Logic Unit, commonly known as the ALU.

Whenever your computer performs a calculation like addition or subtraction, compares two values, or performs logical operations such as AND or OR, the ALU is responsible for carrying out those tasks.

By the end of today's lecture, you'll understand how the ALU works, what operations it performs, how status flags are generated, and why it is essential in every processor.

Let's begin.

2. What is an ALU?

Definition

The Arithmetic Logic Unit (ALU) is a digital circuit inside the CPU that performs arithmetic, logical, comparison, and shift operations on data.

In simple words:

The ALU is the calculator of the CPU.

The ALU does not make decisions about *when* an operation should happen. It performs the operation only after the Control Unit instructs it.

3. Why Do We Need an ALU?

Imagine you write this C++ program:

```
int c = 15 + 25;
```

The compiler converts this into machine instructions.

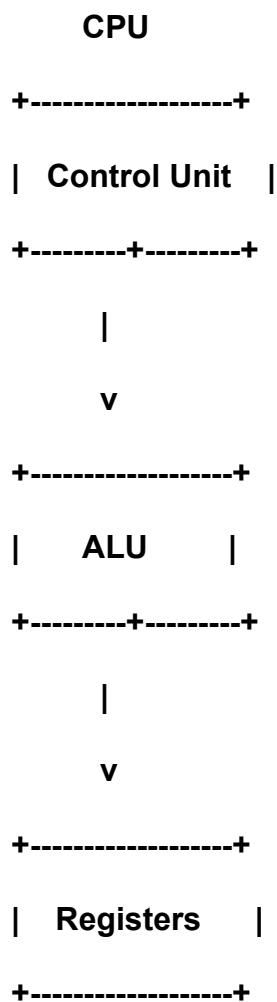
Eventually, the CPU reaches an instruction that tells it to add two values.

The ALU performs the addition.

Without the ALU, the CPU would not be able to:

- Add numbers
 - Subtract numbers
 - Compare values
 - Perform logical operations
-

4. Position of the ALU Inside the CPU



Roles

- Control Unit: Decides *what* operation to perform.
 - Registers: Provide the operands and receive the result.
 - ALU: Executes the requested operation.
-

5. Types of Operations Performed by the ALU

The ALU performs four main categories of operations:

1. Arithmetic Operations
2. Logical Operations
3. Comparison Operations
4. Shift Operations

We'll study each one.

6. Arithmetic Operations

Arithmetic operations involve mathematical calculations.

Addition

Example:

$$15 + 25 = 40$$

Subtraction

$$40 - 10 = 30$$

Multiplication

$$5 \times 8 = 40$$

Division

$$40 \div 5 = 8$$

Modulus (Remainder)

$$17 \% 5 = 2$$

Note: On some processors, multiplication and division are handled by dedicated hardware or specialized execution units rather than the basic ALU. Conceptually, they are part of arithmetic processing.

7. Logical Operations

Logical operations work on binary values (bits).

They are essential for decision-making and bit manipulation.

AND Operation

Rule:

$$1 \text{ AND } 1 = 1$$

Otherwise = 0

Truth Table

A	B	A AND B
---	---	------------

0	0	0
---	---	---

0	1	0
---	---	---

1	0	0
---	---	---

1	1	1
---	---	---

OR Operation

Rule:

If either input is 1

Result = 1

Truth Table

A	B	A OR B
---	---	-----------

0	0	0
---	---	---

0 1 1

1 0 1

1 1 1

NOT Operation

NOT changes:

$0 \rightarrow 1$

$1 \rightarrow 0$

XOR Operation

Rule:

Result is 1 only when the inputs are different.

A B XOR

0 0 0

0 1 1

1 0 1

1 1 0

8. Comparison Operations

The ALU can compare two values.

For example:

$10 == 10$

True

$25 > 15$

True

$5 < 2$

False

These comparisons are used in:

- **if**
- **else**

- `while`
- `for`
- `switch`

and in assembly language branch instructions.

9. Shift Operations

The ALU can shift bits left or right.

Left Shift

00001010

↓

00010100

A left shift by one position often doubles the value for unsigned integers, provided no overflow occurs.

Right Shift

00010100

↓

00001010

A right shift by one position often halves the value for unsigned integers, discarding the least significant bit.

The exact behavior depends on whether the shift is logical or arithmetic and whether the value is signed or unsigned.

10. How the ALU Works

Suppose we execute:

```
int result = 10 + 20;
```

The process is:

Registers

↓

10

20

↓

ALU

↓

30

↓

Register

↓

Memory

The Control Unit controls the overall process, while the ALU performs the addition.

11. Relationship Between CU, Registers, and ALU

Control Unit

↓

Tells ALU what to do

↓

Registers provide data

↓

ALU performs operation

↓

Registers store result

↓

Memory

12. ALU in Assembly Language

Suppose we have this conceptual MIPS example:

add \$t0, \$t1, \$t2

Meaning:

\$t0 = \$t1 + \$t2

The ALU performs the addition.

Another example:

sub \$t0, \$t1, \$t2

The ALU performs subtraction.

For logical operations:

and \$t0, \$t1, \$t2

or \$t0, \$t1, \$t2

xor \$t0, \$t1, \$t2

Again, the ALU performs these operations.

13. Status Flags (Flags Register)

After many ALU operations, the processor updates status flags.

These flags describe information about the result.

Common flags include:

- **Zero Flag (Z)**
- **Carry Flag (C)**
- **Sign/Negative Flag (S or N)**
- **Overflow Flag (V)**

Different processor architectures may use different names and additional flags.

Zero Flag (Z)

Set when the result is zero.

Example:

$$5 - 5 = 0$$

Zero Flag = 1

Carry Flag (C)

Often set when an unsigned arithmetic operation produces a carry out (addition) or requires a borrow (subtraction, depending on the architecture).

Example (4-bit addition):

1111

+

0001

=

10000

A carry is generated beyond the available bits.

Sign (Negative) Flag

Indicates that the result is negative in signed arithmetic.

Example:

$$5 - 10 = -5$$

The processor marks the result as negative according to its signed representation.

Overflow Flag (V)

Indicates that a signed arithmetic result cannot be represented in the available number of bits.

Example:

$$127 + 1$$

In an 8-bit signed representation, this causes an overflow.

14. Complete ALU Diagram

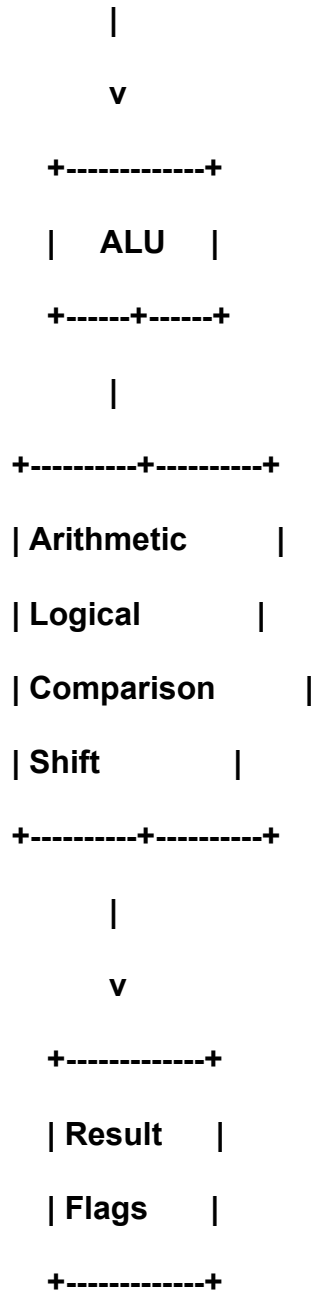
Registers

+-----+

| Operand A |

| Operand B |

+-----+



15. Real-Life Analogy

Imagine a calculator.

You enter:

25 + 35

The calculator:

- Receives the numbers.
- Receives the operation.
- Computes the answer.
- Displays the result.

The ALU works in a similar way.

The only difference is that the ALU receives instructions from the Control Unit, not directly from a human.

16. Practical Example

Suppose:

int x = 18;

int y = 12;

int z = x - y;

Execution:

RAM

↓

Registers

↓

18

12

↓

ALU

↓

6

↓

Register

↓

RAM



Lecture Summary

Today, we explored the Arithmetic Logic Unit (ALU), one of the most important components inside the CPU.

We learned that the ALU performs arithmetic operations like addition and subtraction, logical operations such as AND, OR, NOT, and XOR, comparison operations, and shift operations.

We also studied how the ALU works together with the Control Unit and Registers to execute instructions.

Finally, we introduced status flags—including the Zero, Carry, Sign, and Overflow flags—which provide information about the result of ALU operations.

In the next lecture, we'll study the Control Unit (CU) in detail and learn how it fetches, decodes, and coordinates the execution of instructions throughout the CPU.

Thank you for watching, and I'll see you in the next lecture.
